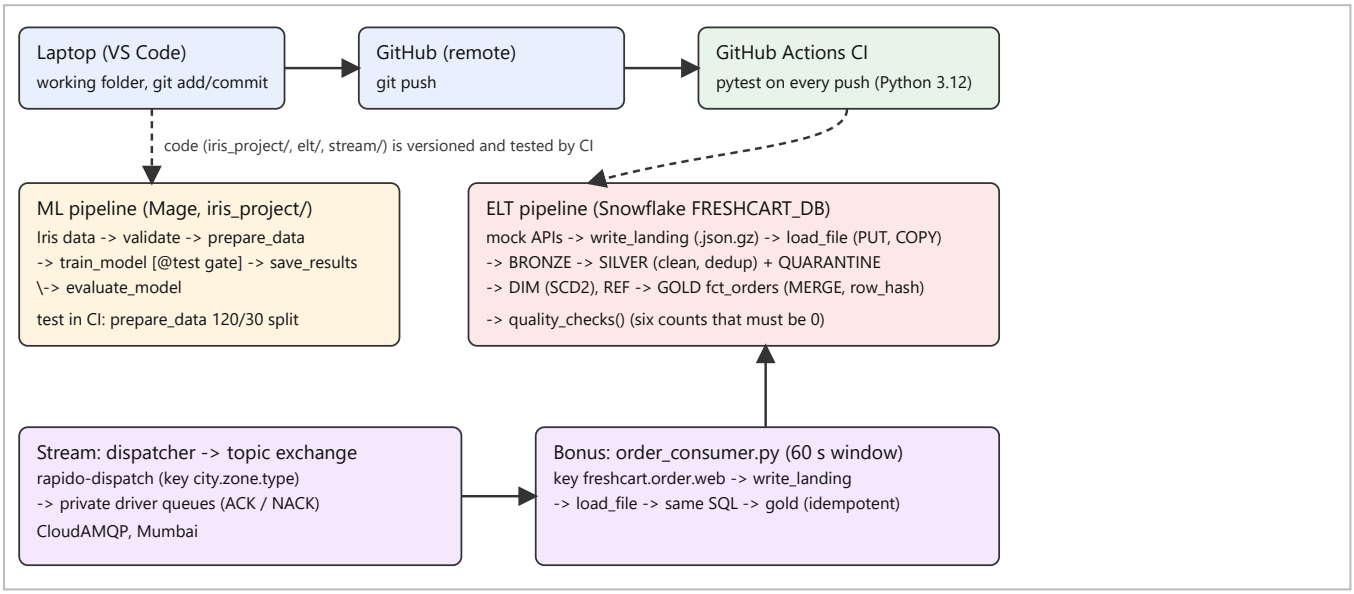
From Git to Gold: Pipeline Orchestration

Name: Gunti Ravi Kumar | Roll no: 23BCS095 | B.Tech, Computer Science and Engineering | Course: Data Engineering
Repository: https://github.com/Ravikumar-2016/freshcart-pipelines (public repository)
Environment: Windows, VS Code, Python 3.12.6, Snowflake, GitHub Actions, Mage AI 0.9.80, CloudAMQP (RabbitMQ). Evidence files are in `report/screenshots/`; screenshots are in the appendix. The project was built with the help of Claude Code (an AI coding assistant).

Integration diagram



Code lives in Git; CI tests the Mage `prepare_data` block on every push. The ELT pipeline produces the gold table. The Mage pipeline trains on the Iris dataset from scikit-learn (it does not read the gold table). The stream delivers requests to drivers (Part D); the bonus consumer feeds order events from the same broker into bronze and so into gold.

Part A: Git and CI/CD

A1 Repository and commit history

The repository has the required structure: `README.md`, `.gitignore` (ignores `.env`, `*.pkl`, `*.csv`, `mage.log`, `__pycache__/` and `.venv/`), `requirements.txt`, `elt/`, `iris_project/`, `stream/`, `tests/test_prepare_data.py`, `.github/workflows/ci.yml`. Output of `git log --oneline`:

```
(snapshot taken when HEAD was 51e829a; later commits are not listed)
51e829a docs: final audit, corrected report, lecture-backed ETL answer, regenerated PDF
a1af458 chore: finalize data engineering assignment submission
e0e1f5d feat(stream): complete RabbitMQ routing, ACK/NACK and FreshCart bonus integration
4d59385 feat(stream): implement and verify RabbitMQ routing and ACK/NACK (D1-D3)
3e385f3 feat(elt): add future-date quality check with failure demo and verified cleanup (B5)
f83bf2f feat(elt): add CUST-02 Gold tier event and verify SCD2/MERGE idempotency (B4)
72e290d docs(elt): record B4 pre-run predictions and state
db8cd3c feat(elt): add unsupported_currency validation rule (B3)
af79117 feat(elt): FreshCart ELT notebook, SQL scripts and B1 baseline on Snowflake
1428aef feat(iris): add Mage iris_classifier pipeline with tests, evaluation branch and validation block
a55d012 docs: add Part A CI evidence and answers
0c83c3e fix: restore correct 80/20 train test split
b038cf1 test: demonstrate failing CI for incorrect split
6d86300 ci: add GitHub Actions workflow
4d1e80a feat: add Iris data transformer and tests
4ef2ad1 chore: initial repository structure and .gitignore
```

A2 Where does each command act?

Command	Where the change happens	What exactly happens?
`git init`	working folder / local repository	Makes the current folder a Git repository by creating a hidden `.git` folder; the repository is still empty.
`git add train.py`	working folder -> staging area	Copies the current version of `train.py` into the staging area, the list of changes that will go into the next commit.

`git commit -m "msg"`	staging area -> local repository	Saves everything in the staging area as one commit with the message "msg", in the repository on this computer.
`git remote add origin URL`	local repository (settings only)	Gives the GitHub address the short name `origin`; it only stores the name and sends nothing.
`git push -u origin main`	local repository -> remote (GitHub)	Sends the commits of the local `main` branch to GitHub and remembers `origin/main`, so a plain `git push` works afterwards.
`git pull origin main`	remote -> local repository and working folder	Gets new commits from `origin/main` on GitHub and merges them into the local branch and the files.

After `git commit`, can a teammate see the code on GitHub? No. `git commit` only saves the snapshot in the local repository (box 2 of the four-box flow). GitHub is the remote repository (box 3) and receives commits only after `git push`. In this project the first commit existed locally while GitHub was still empty, until the first push.

A3 CI workflow

`tests/test_prepare_data.py` checks that `prepare_data.transform` gives 120 train and 30 test rows (`train_test_split(test_size=0.2, random_state=42)`). `.github/workflows/ci.yml` runs on every push: check out the code, set up Python 3.12, `pip install -r requirements.txt`, `python -m pytest`. Observed runs: CI #1 green (`6d86300`); CI #2 red after `test_size` was set to 0.3 (`b038cf1`; locally the test failed with `105 == 120`); the fix `0c83c3e` gave CI #3 green. Screenshots are in the appendix. The workflow run for the later commit `a1af458` (CI #6) was also green; this was checked in the GitHub Actions tab and no screenshot of that run is included.

Part B: FreshCart ELT on Snowflake

Notebook `elt/freshcart_elt.ipynb`; SQL in `elt/sql/`; code in `elt/freshcart.py`; sample data in `elt/mock_data.py` (the course sample data; there are no real APIs). The step-by-step work (B1 to B5) was done in database `FRESHCART_DB`; the final notebook was then executed once, top to bottom, in a fresh database `FRESHCART_FINAL` so that the B1 numbers are exact.

B1 Baseline (one run on an empty database)

Check	Expected	Observed
Raw order rows in bronze	8 (6 web + 2 mobile)	8
Rows passing cleaning	6	6
Quarantine	2 (ORD-1004, ORD-1006)	2 (same orders)
After deduplication	4	4
Gold line	4 orders, 7,785.00, 126 points	4 orders, 7785.00, 126 points
Rows rejected	25%	25%

B2 Trace of ORD-1003 and ORD-1005

ORD-1003, step 1: the raw records. `bronze.orders_raw` holds two rows for this order. The web row (ingest_seq 2) is flat JSON: `order_id, cust_email, amount: null, order_date "2026-09-02", updated_at "2026-09-02 09:10:00"`. The mobile row (ingest_seq 8) is nested JSON with camelCase names: `orderId, customer.email, total.value "2,100.00", orderDate "02/09/2026", updatedAt "2026-09-02T18:40:00"`. The names differ because the two rows come from two different systems (the website export and the mobile API). Bronze keeps each record exactly as received, so nothing is lost.

Step 2: the same columns in `silver.orders_checked`. In `10_orders_clean.sql` the `unified` CTE reads the web names (`raw:order_id, raw:cust_email, raw:amount, raw:updated_at`) and the mobile names (`raw:orderId, raw:customer.email, raw:total.value, raw:updatedAt`) into one set of column names. The `typed` CTE then cleans them: `LOWER(TRIM(email))` turns `Meera@Outlook.com` into `meera@outlook.com`; `INITCAP` fixes the name; `COALESCE(TRY_TO_DATE(..'YYYY-MM-DD'), TRY_TO_DATE(..'DD/MM/YYYY'), ...)` reads `02/09/2026` as 2 September 2026; `TRY_TO_DECIMAL(REGEXP_REPLACE(amount,'[^0-9.-]',''),12,2)` removes the comma so `"2,100.00"` becomes 2100.00 (the web NULL stays NULL); `TRY_TO_TIMESTAMP_NTZ` reads `updated_at`. Both copies then have the same email, name, date, currency and pincode and differ only in amount (NULL or 2100.00) and `updated_at`. Both pass validation, because a missing amount is allowed.

Step 3: which copy survives. `20_orders_dedup.sql` uses `QUALIFY ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY updated_at DESC, ingest_seq DESC) = 1`. The copies of one order are numbered from newest to oldest and only number 1 is kept. The mobile copy (18:40) is newer than the web copy (09:10), so it gets number 1 and survives with amount 2100.00. (If two copies had the same time, the later arrival, the higher `ingest_seq`, would win.)

Step 4: gold. The INR rate is 1.0, so `amount_inr = ROUND(2100.00 x 1.0, 2) = 2100.00`. Meera is Platinum, multiplier 3, so `loyalty_points = FLOOR(2100.00 / 100) x 3 = 21 x 3 = 63`.

ORD-1005 and ORD-1001 (Priya). The tier is looked up for the date of the order. The table `dim.customers_scd2` has two rows for Priya: Silver from 2023-04-10 to 2026-09-02 and Gold from 2026-09-03. In `40_gold.sql` the join is `c.email = o.email AND o.order_date BETWEEN c.valid_from AND c.valid_to`, followed by `ref.tier_multiplier` (Silver 1, Gold 2, Platinum 3). ORD-1001 is dated 2026-09-01, which falls in the Silver period: `FLOOR(1250/100) x 1 = 12` points. ORD-1005 is dated 2026-09-03, which falls in the Gold period: `FLOOR(899/100) x 2 = 8 x 2 = 16` points.

B3 Rule `unsupported_currency`

One line was added inside `ARRAY_CONSTRUCT_COMPACT` in `10_orders_clean.sql`: `IFF(currency IS NULL OR currency NOT IN ('INR','USD'), 'unsupported_currency', NULL)`. The currency is already upper-cased there, so `inr` and `usd` still pass. ORD-1006 has currency EUR. Its reasons **before** the change were `["bad_email","bad_pincode"]`; **after** the re-run, in `silver.orders_checked`, they are `["bad_email","bad_pincode","unsupported_currency"]`.
Does the gold total change? No (4 orders, 7,785.00, 126 points before and after). ORD-1006 was already rejected because of its email and pincode, so it never reached `silver.orders_clean` or gold; the new rule only adds a third reason to a row that was already out. No INR or USD order lost a place.
Does the re-run add a second quarantine row for ORD-1006? No. After the re-run `quarantine.orders` still has one row for ORD-1006, although bronze and `silver.orders_checked` now hold two copies of it. The part of the SQL that decides this is the `AND NOT EXISTS (...)` in the quarantine insert of `10_orders_clean.sql`: a rejected row is skipped when `quarantine.orders` already has the same `source`, `order_id` and `updated_at`. The check does not compare the reasons, so the old quarantine row keeps its old reasons and the new reason is visible only in `silver.orders_checked`.

B4 CUST-02 becomes Gold on 2026-08-15

The event was added to `MOCK_CRM_TIER_EVENTS`. **Predictions** were written down and committed (commit `72e290d`) before the pipeline was run. In `dim.customers_scd2` every tier change starts a new period that lasts until the day before the next change, so CUST-02 (Silver from 2024-01-15) should get two rows. ORD-1002 is dated 2026-09-01, after 2026-08-15, so it should use Gold (multiplier 2). Its amount is 3536.00, so the points should be `FLOOR(3536/100) x 2 = 70` instead of 35, the total should rise by 35 to 161, and only ORD-1002 should change. **Observed results** after the run:

Quantity	Prediction	Observed
CUST-02 rows in `dim.customers_scd2`	2	2 (Silver to 2026-08-14, Gold from 2026-08-15)
Tier used for ORD-1002	Gold	Gold
Loyalty points for ORD-1002	70	70
Total points in gold	161	161
Gold rows whose `published_at` changed	1	1 (ORD-1002 only)

A second run of the transformation changed no `published_at` and no `row_hash`. **Why gold is safe to re-run:** each run rebuilds `silver.orders_enriched` and `MERGE`s it into `gold.fct_orders` by `order_id`. `row_hash` is a `HASH` of all columns except `order_id`. The MERGE updates a row, and sets a new `published_at`, only when the hash is different (`WHEN MATCHED AND t.row_hash <> s.row_hash`), and it inserts only orders that are not in gold yet. So a re-run never creates a second row for an order and never touches a row whose values are the same. **Why bronze still grows by 8 rows each run:** every run extracts the same 8 raw orders again, writes them to new landing files and `COPY INTO` adds them. Bronze is an append-only copy of everything that was received and does no deduplication; the duplicates are removed later in silver by the `ROW_NUMBER` step.

B5 Sixth check and ELT vs ETL

The sixth check in `quality_checks()` is `SELECT COUNT_IF(order_date > CURRENT_DATE()) FROM silver.orders_dedup`, printed as "orders dated in the future"; it is 0 on the real data. To make it fail, a mock order `TEST-B5-FUTURE-DATE-001` dated 2026-12-31 was loaded. It has no amount, so no other check is triggered (a missing amount is allowed and a future date has no FX rate). The output was `orders dated in the future 1 PROBLEM`. The cleanup first counted the exact markers and checked them (bronze 1, gold 1, quarantine 0, staged file 1, other rows not matched), then deleted only those rows and the staged file. After the transform was run again the check was back to `0 OK` and all tables matched the state before the test (19 of 19 checks).
ELT vs ETL. FreshCart's orders come from two systems in different shapes and with errors (flat web JSON, nested mobile JSON, duplicates, bad emails). With ELT the raw rows are first loaded unchanged into bronze and then cleaned with SQL inside Snowflake. This keeps the original data, so a changed rule can be run again without fetching the data again (B3 and B4 did not need new extraction), and rejected rows stay in quarantine with their reasons. ETL would still be better when sensitive data such as card or Aadhaar numbers must be masked or removed before it is stored in the warehouse, because in ELT the raw data lands first (the course notes, "Data transformation and orchestration", 30-09-2026, section "ETL and ELT: where the transformation runs", list strict pre-load compliance, small or costly targets and heavy non-SQL processing as the cases where ETL fits best).

Part C: ML pipeline with Mage

C1 Graph and run

```
load_iris (data_loader, @test: 150 rows, target)
  -> validate_data (transformer, C4, @test x2: no missing values, 3 classes)
  -> prepare_data (transformer, @test: split column)
  -> train_model (transformer, @test: accuracy >= 0.90, 30 test rows)
       |-> save_results (data_exporter, @test: CSV with 30 rows)
       '-> evaluate_model (transformer, C3, @test: 3 classes, support 30)
```

`mage run iris_project iris_classifier --test`: every block printed `1/1 tests passed` (2/2 for `validate_data`), accuracy 1.0 at the default `max_depth` 3, `iris_predictions.csv` was written and the exit code was 0 (`report/screenshots/C/C1_mage_run_test.txt` and the executed notebook `iris_project/iris_classifier_run.ipynb`).

C2 Tests as a gate

Each run used `--runtime-vars '{"max_depth": N}'` after `iris_predictions.csv` had been deleted. The assignment gives no accuracy threshold; the 0.90 in the `train_model` test is a choice made for this project, because the reference Mage notebook was not available, so the pass/fail column depends on it.

max_depth	Test accuracy	test_accuracy passed?	CSV created?
1	0.6333	No (exit 1)	No
2	0.9667	Yes	Yes
3	1.0000	Yes	Yes
4	1.0000	Yes	Yes
5	1.0000	Yes	Yes

What the numbers mean. There are 30 test rows. At depth 1 the tree can make only one split, so it cannot separate three classes: 19 of 30 rows were right (0.6333). At depth 2 it got 29 of 30 right (0.9667; the confusion matrix in C3 shows one versicolor predicted as virginica), and from depth 3 on all 30 were right. **How a failing `@test` stops `save_results`:** with `--test` Mage runs the tests of a block right after the block. At depth 1 the `train_model` test failed (0.6333 is below 0.90), Mage stopped the run with exit code 1 and the blocks after `train_model` never started: the log has no "Saved" line and no confusion matrix, and the CSV did not exist. **Why this is better than saving first and checking later:** a bad result never exists as a file that something else could pick up, and the failure shows immediately in the run.

C3 Evaluation branch

`train_model` has two downstream blocks, `save_results` and `evaluate_model`. Relevant part of `iris_project/pipelines/iris_classifier/metadata.yaml` (other keys omitted):

```
- name: train_model        type: transformer
  upstream_blocks:   [prepare_data]
  downstream_blocks: [save_results, evaluate_model]
- name: evaluate_model     type: transformer
  upstream_blocks:   [train_model]
  downstream_blocks: []
```

At `max_depth` 2 the confusion matrix (rows actual, columns predicted) is: setosa [10, 0, 0]; versicolor [0, 8, 1]; virginica [0, 0, 11]. Metrics (precision / recall): setosa 1.0 / 1.0; versicolor 1.0 / 0.8889; virginica 0.9167 / 1.0; accuracy 0.9667.
UI screenshot through the Cloudflare tunnel. Mage was started locally (`mage start iris_project`, port 6789) and made reachable with a Cloudflare Quick Tunnel (`cloudflared`). Figure C3 in the appendix is a browser screenshot with the public address `determination-unwrap-donated-dylan.trycloudflare.com/pipelines/iris_classifier/triggers` in the address bar. It shows the `iris_classifier` pipeline graph (the graph is on the right of the pipeline's Triggers page) with all six blocks and the branch from `train_model` to `save_results` and `evaluate_model`. A second screenshot, taken on the local server, is kept as additional local evidence. The first attempts to start the tunnel failed (`cloudflared` reported "context deadline exceeded", and direct requests to Cloudflare's tunnel API returned HTTP 429 or 500 while ordinary HTTPS to the same host worked), which pointed to a temporary problem on Cloudflare's side; the tunnel worked later the same day (log of the failed attempts: `report/screenshots/C/C3_cloudflare_tunnel_probe.txt`). No `cloudflared` process was running when this was checked afterwards.

C4 Own block

`validate_data` is a transformer between `load_iris` and `prepare_data` with two `@test` functions (no missing values, exactly 3 classes). The full six-block run passed. When its tests were run directly on bad data they failed as intended (one NaN: "Found 1 missing values"; two classes: "Expected exactly 3 classes").

Part D: Food-order stream with RabbitMQ

D1 Routing

Exchange `rapido-dispatch` is a topic exchange; every driver has its own queue bound with its own pattern. In a pattern `*` matches exactly one word and `#` matches zero or more words. Bindings: Ravi `bangalore.koramangala.*`, Priya `bangalore.indiranagar.*`, Arjun `bangalore.*.ride`, Meena `bangalore.*.food`, Karthik `chennai.tnagar.food`, Control `bangalore.#`.

Predictions (on paper), with the reason; **observed** = what `stream/verify_routing.py` read back from the broker's queues after publishing each key. *Note on timing: the expected recipients were worked out from the binding patterns before the broker test, but Git has the predictions and the test logs in the same commit (`4d59385`), so the commit history alone does not show which came first.*

#	Routing key	Predicted queues and why	Observed
1	bangalore.koramangala.food	Ravi (* = food), Meena (* = koramangala), Control (# = koramangala.food)	Ravi, Meena, Control
2	bangalore.indiranagar.ride	Priya, Arjun (* = indiranagar), Control	Priya, Arjun, Control
3	bangalore.whitefield.ride	Arjun (* = whitefield), Control; nobody has a whitefield pattern	Arjun, Control
4	chennai.tnagar.food	Karthik (exact match); the others start with bangalore	Karthik
5	chennai.whitefield.ride	dropped: the only chennai binding is chennai.tnagar.food	dropped (the broker reported it as unroutable)
6	bangalore.koramangala	Control only (see below)	Control

Question 6, the two-word key. `bangalore.koramangala` has only two words. The patterns of Ravi, Priya, Arjun and Meena each have three words, and each contains a `*` that must match exactly one word; the key has no third word, so they cannot match. Karthik's pattern is a three-word literal, so it cannot match either. Only `bangalore.#` can match, because `#` may match zero or more words: `bangalore` matches the first word and `#` matches the remaining word `koramangala` (it would match even if there were no remaining word). So only Control gets the message. All six observed results were equal to the predictions.

D2 Dispatcher and drivers

`stream/dispatcher.py` sends a request every 2 seconds: it picks a city (bangalore or chennai), a zone and a type (ride or food), builds the key `city.zone.type` and publishes the JSON `{id, city, zone, type, ts}` as a persistent message to `rapido-dispatch`. `stream/driver.py NAME PATTERN` creates its own private queue, binds it with PATTERN, prints each request it gets (name, key, payload) and sends `basic_ack` after processing it. Six drivers and the dispatcher ran as separate processes; the dispatcher published from 05:23:59 to 05:26:00 UTC (about 2 minutes), 59 messages. A script compared the logs with the routing rules for every message: each message reached exactly the drivers that the rules predict and no other driver. Counts: Ravi 13, Priya 7, Arjun 17, Meena 15, Karthik 2, Control 32, all acknowledged; 25 messages matched no pattern and were dropped by the exchange. Evidence: `report/screenshots/D/D2_*` (log files). Example, message #2 (`bangalore.koramangala.ride` matches Ravi, Arjun and Control only):

```
SENT #002 id=req-0002-242d4e key=bangalore.koramangala.ride
[Control] RECEIVED key=bangalore.koramangala.ride id=req-0002-242d4e payload={"id":"req-0002-
242d4e","city":"bangalore","zone":"koramangala","type":"ride","ts":"2026-11-12T05:24:01+00:00"} -> ACK
[Arjun]   RECEIVED ... id=req-0002-242d4e ... -> ACK      [Ravi] RECEIVED ... id=req-0002-242d4e ... -> ACK
(Priya, Meena, Karthik: nothing)
```

D3 ACK and NACK

In `driver.py` a message without `zone` gets `basic_nack(requeue=False)`; every other message gets `basic_ack`. In `dispatcher.py` every 10th message is sent without the `zone` field. In a live run of 35 messages, messages #10, #20 and #30 were malformed. #10 and #20 matched drivers: Arjun and Control printed `REJECTED ... missing field 'zone' -> NACK (requeue=False, discarded)` for both, and every valid message was processed and ACKed; #30 matched no queue. A separate script (`stream/d3_demo.py`) read the queue depth from the broker: a valid message was ACKed and the depth went from 1 to 0; a malformed one was NACKed with `requeue=False` and the depth went from 1 to 0 (the message is gone). No dead-letter exchange was configured, so a rejected message is simply deleted.

What happens if the driver crashes before sending the ACK? With manual acknowledgements RabbitMQ keeps a delivered message as "unacknowledged" until the ACK arrives. If the connection closes first, the broker puts the message back in the queue and delivers it again, marked `redelivered=True`. The demo showed it: after a consumer closed its channel without ACK the queue depth was 1 again and the next delivery had `redelivered=True`. So no message is lost, but it can be processed twice. **What would `requeue=True` do for a message that is always bad?** It goes back to the queue, is delivered again, fails again, and this repeats without end (a poison message). In the demo the bad message

came back on 4 of 4 attempts with `redelivered=True` and the depth stayed at 1; in a real system it wastes CPU and blocks the messages behind it. With `requeue=False` it is removed after the first failure.

Bonus: stream into the lakehouse

Flow. `stream/order_consumer.py` listens on a durable queue bound to the same exchange with the key `freshcart.order.web` (no D1 driver pattern matches it). It collects order messages in the web format for 60 seconds. A message that is not valid JSON or lacks `order_id`, `cust_email` or `order_date` is NACKed with `requeue=False`. The collected messages are written to a raw file by the existing `write_landing()`, loaded into bronze by the existing `load_file()` (PUT and COPY INTO), and the existing transform SQL builds silver and gold, ending in `gold.fct_orders`, followed by `quality_checks()`. The messages are acknowledged only after the file is loaded into bronze; if writing or loading fails they are requeued, so nothing is lost. **Test run:** three marked orders `BONUS-TEST-001..003`, a repeat of 001 and one malformed message were sent in each of two runs:

	Baseline	After run 1	After run 2
Messages collected / rejected		4 / 1	4 / 1
Gold orders	4	7	7
Gold revenue (INR)	7,785.00	9,535.00	9,535.00
Gold points	161	205	205
bronze order rows	24	28	32

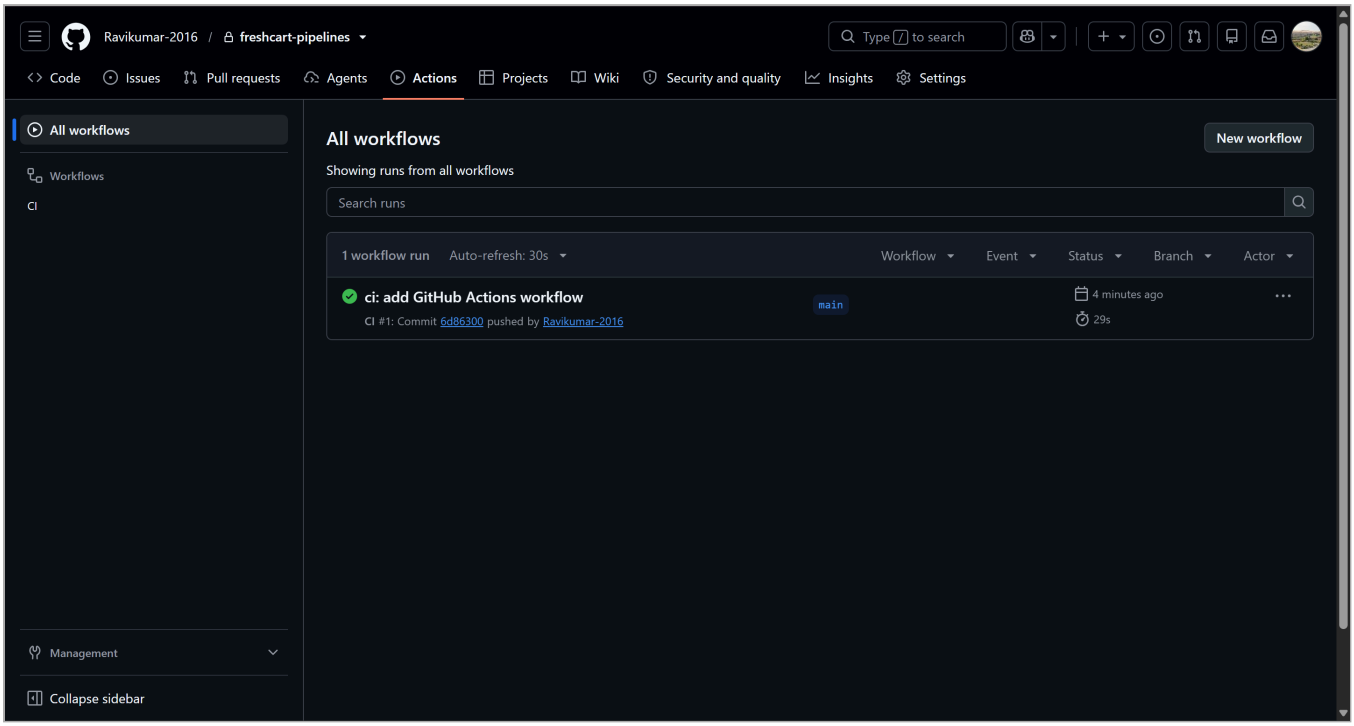
Result. Run 1 added the three orders (+1,750.00 and +44 points, as expected from the tiers). Run 2 sent the same orders again: bronze grew by 4 rows, but gold stayed at 7 orders, 9,535.00 and 205 points and every `published_at` was unchanged. The reason is the same as in B4: silver keeps one row per order (`QUALIFY ROW_NUMBER`) and the gold MERGE changes a row only when its `row_hash` changes. Afterwards the test data was removed (exact marker counts were checked first) and the final state equalled the baseline (`report/screenshots/Bonus/`).

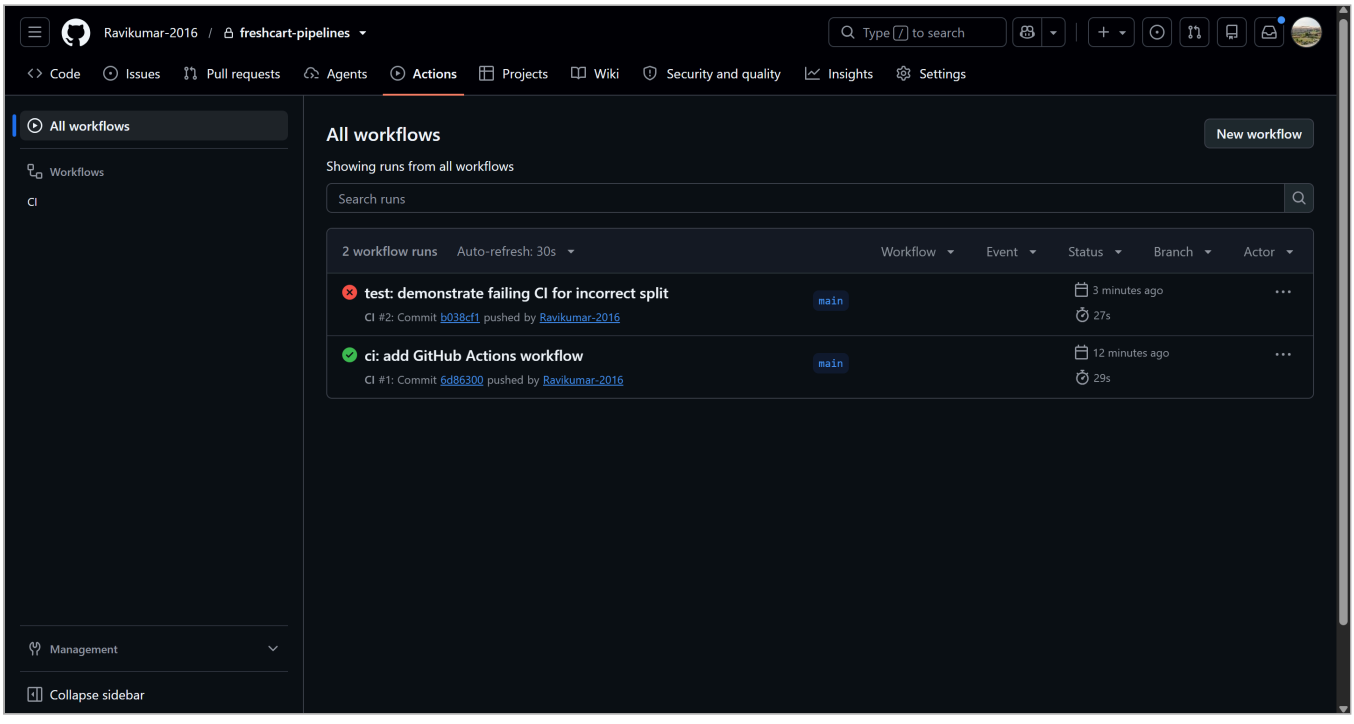
Reflection

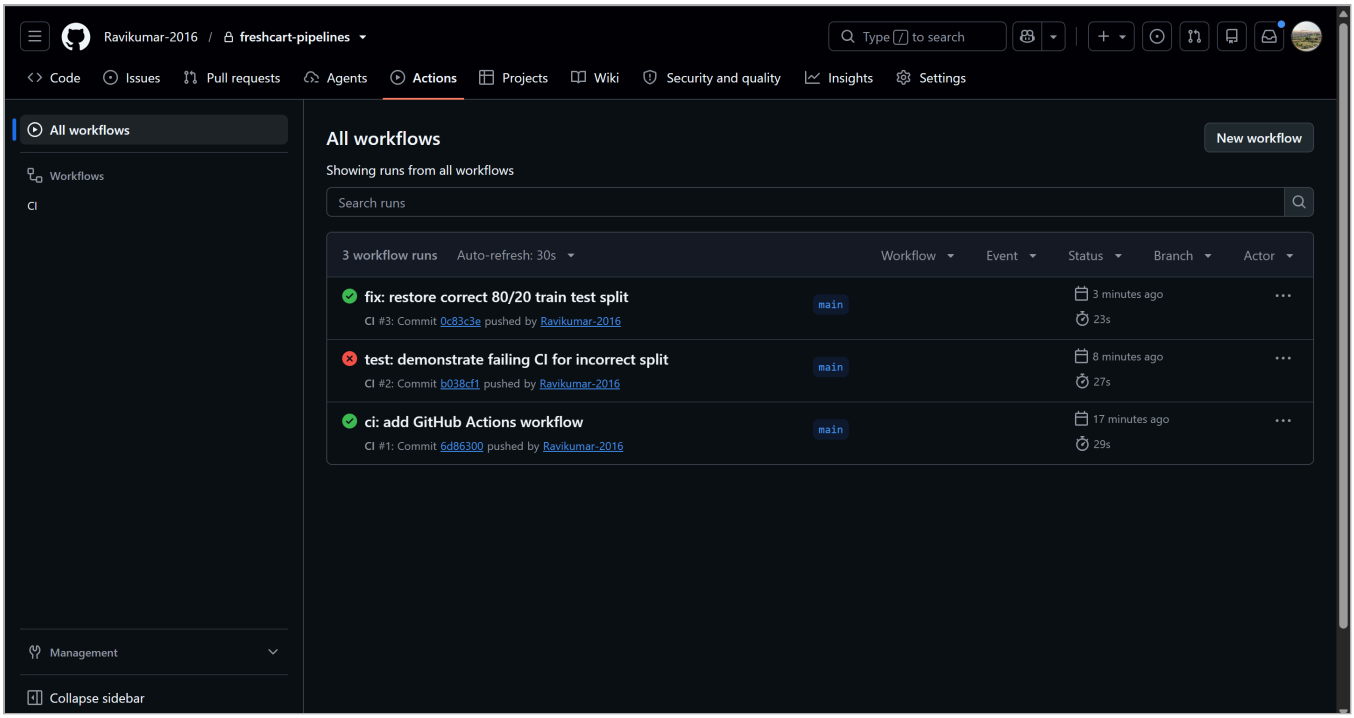
One thing that broke: in the B4 part of the notebook, the analysis cell stopped with `KeyError: 'order_id'`, after the pipeline cell above it had already run once with the new CRM event. The traceback pointed at a merge of two data frames, and the cause was that Snowflake returns column names in upper case (`ORDER_ID`) while the cell used lower case. Running the pipeline again would have added another 8 rows to bronze and spoiled the "before" comparison, so it was not run again. Instead the "before" state was read with Snowflake Time Travel (the tables as they were just before the run), after checking that it matched the "before" snapshot saved earlier, and the column names were corrected. This was found and fixed with the help of Claude Code, the AI assistant used for the project. Later, while preparing the final run, it turned out that the notebook still depended on that one-off history (a fixed Time Travel time, and the B4 event already inside the sample data), so a clean top-to-bottom run would not have reproduced the B1 numbers. This was fixed by keeping the event separate until section B4, removing the fixed time, and executing the whole notebook once in a fresh database (`FRESHCART_FINAL`). The lesson: save a "before" snapshot first, and make a notebook re-runnable from the start.

Appendix: screenshots and evidence

A3: CI runs (green, red, green again)






Ravikumar-2016 / freshcart-pipelines
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
All workflows
Workflows
CI
Management
Collapse sidebar
All workflows
New workflow
Showing runs from all workflows
Search runs
3 workflow runs
Auto-refresh: 30s
Workflow
Event
Status
Branch
Actor
fix: restore correct 80/20 train test split
CI #3: Commit 0c83c3e pushed by Ravikumar-2016
main
3 minutes ago
23s
test: demonstrate failing CI for incorrect split
CI #2: Commit b038cf1 pushed by Ravikumar-2016
main
8 minutes ago
27s
ci: add GitHub Actions workflow
CI #1: Commit 6d86300 pushed by Ravikumar-2016
main
17 minutes ago
29s

B1: baseline run (rendered notebook outputs)

2	3	web	{\n "amount": "42.50",\n "currency": "USD",\...
3	4	web	{\n "amount": "-500",\n "currency": "INR",\n...
4	5	web	{\n "amount": "₹ 899",\n "currency": "inr",\...
5	6	web	{\n "amount": 30,\n "currency": "EUR",\n "c...
6	7	mobile	{\n "customer": {\n "email": "arjun.k@yaho...
7	8	mobile	{\n "customer": {\n "email": "Meera@Outloo...

Silver: cleaned rows (6)

In [9]:
```
fc.show("SELECT ingest_seq, source, order_id, email, name, order_date, amount, currency, pincode, amount_missing FROM silver.orders_clean ORDER BY ingest_seq")
```

Out[9]:

	INGEST_SEQ	SOURCE	ORDER_ID	EMAIL	NAME	ORDER_DATE	AMOUNT	CURRENCY	PINCODE	AMOUNT_MISSING
0	1	web	ORD-1001	priya.r@gmail.com	Priya R	2026-09-01	1250.00	INR	600041	False
1	2	web	ORD-1003	meera@outlook.com	Meera S	2026-09-02	None	INR	400050	True
2	3	web	ORD-1002	arjun.k@yahoo.in	Arjun K	2026-09-01	42.50	USD	560034	False
3	5	web	ORD-1005	priya.r@gmail.com	Priya R	2026-09-03	899.00	INR	600041	False
4	7	mobile	ORD-1002	arjun.k@yahoo.in	Arjun K	2026-09-01	42.50	USD	560034	False
5	8	mobile	ORD-1003	meera@outlook.com	Meera S	2026-09-02	2100.00	INR	400050	False

Quarantine: rejected rows and why

In [10]:
```
fc.show("SELECT run_id, source, order_id, reasons FROM quarantine.orders ORDER BY order_id")
```

Out[10]:

	RUN_ID	SOURCE	ORDER_ID	REASONS
0	20261112_040919	web	ORD-1004	[\n "non_positive_amount"\n]
1	20261112_040919	web	ORD-1006	[\n "bad_email",\n "bad_pincode"\n]

Silver: one row per order (4)

In [11]:
```
fc.show("SELECT order_id, ingest_seq, updated_at, amount FROM silver.orders_dedup ORDER BY order_id")
```

Out[11]:

	ORDER_ID	INGEST_SEQ	UPDATED_AT	AMOUNT
0	ORD-1001	1	2026-09-01 10:02:00	1250.00
1	ORD-1002	7	2026-09-01 11:15:00	42.50
2	ORD-1003	8	2026-09-02 18:40:00	2100.00
3	ORD-1005	5	2026-09-03 16:45:00	899.00

Dim: customer tier history (SCD2)

In [12]:
```
fc.show("SELECT * FROM dim.customers_scd2 ORDER BY cust_id, valid_from")
```

Out[12]:

	CUST_ID	EMAIL	TIER	VALID_FROM	VALID_TO	IS_CURRENT
0	CUST-01	priya.r@gmail.com	Silver	2023-04-10	2026-09-02	False
1	CUST-01	priya.r@gmail.com	Gold	2026-09-03	9999-12-31	True
2	CUST-02	arjun.k@yahoo.in	Silver	2024-01-15	9999-12-31	True
3	CUST-03	meera@outlook.com	Platinum	2022-11-01	9999-12-31	True

Ref: daily FX rates, weekends filled

In [13]:
```
fc.show("SELECT * FROM ref.fx_daily WHERE currency = 'USD' ORDER BY rate_date LIMIT 10")
```

Out[13]:

	CURRENCY	RATE_DATE	RATE_TO_INR	IS_CARRIED_FORWARD
0	USD	2026-09-01	83.2000	False
1	USD	2026-09-02	83.3500	False
2	USD	2026-09-03	83.1000	False
3	USD	2026-09-04	83.1000	True
4	USD	2026-09-05	83.1000	True
5	USD	2026-09-06	83.1000	True
6	USD	2026-09-07	83.1000	True
7	USD	2026-09-08	83.1000	True
8	USD	2026-09-09	83.1000	True
9	USD	2026-09-10	83.1000	True

Ref: pincode geography

In [14]:
```
fc.show("SELECT * FROM ref.pincode_geo ORDER BY pincode")
```

Out[14]:

	PINCODE	CITY	STATE	REGION	UPDATED_AT
0	400050	Mumbai	Maharashtra	West	2026-11-11 20:09:27.889000-08:00
1	560034	Bengaluru	Karnataka	South	2026-11-11 20:09:27.889000-08:00
2	600041	Chennai	Tamil Nadu	South	2026-11-11 20:09:27.889000-08:00

Gold: the final table

In [15]:
```
fc.show("SELECT order_id, order_date, tier, amount, currency, fx_rate, amount_inr, loyalty_points, is_high_value, city, region FROM gold.fct_orders ORDER BY order_i
```

Out[15]:

	ORDER_ID	ORDER_DATE	TIER	AMOUNT	CURRENCY	FX_RATE	AMOUNT_INR	LOYALTY_POINTS	IS_HIGH_VALUE	CITY	REGION
0	ORD-1001	2026-09-01	Silver	1250.00	INR	1.0000	1250.00	12	False	Chennai	South
1	ORD-1002	2026-09-01	Silver	42.50	USD	83.2000	3536.00	35	True	Bengaluru	South
2	ORD-1003	2026-09-02	Platinum	2100.00	INR	1.0000	2100.00	63	True	Mumbai	West
3	ORD-1005	2026-09-03	Gold	899.00	INR	1.0000	899.00	16	False	Chennai	South

Gold totals

In [16]:
```
fc.show("SELECT COUNT(*) AS orders, SUM(amount_inr) AS revenue_inr, SUM(loyalty_points) AS points FROM gold.fct_orders")
```

Out[16]:

	ORDERS	REVENUE_INR	POINTS
0	4	7785.00	126

10. B1 check: expected vs actual (after exactly one run)

Expected values come from the assignment table (B1) and the flowchart. `PASS` only if the actual number matches.

In [17]:
```
q = lambda sql: fc.query(sql)[0][0]
rejected_share = float(q("SELECT COUNT_IF(ARRAY_SIZE(reasons) > 0) / NULLIF(COUNT(*), 0) FROM silver.orders_checked"))
quarantined = sorted(r[0] for r in fc.query("SELECT order_id FROM quarantine.orders"))
gold_rev = q("SELECT SUM(amount_inr) FROM gold.fct_orders")
rows = [
    ("Raw order rows in bronze.orders_raw", 8, q("SELECT COUNT(*) FROM bronze.orders_raw")),
    ("  of which web", 6, q("SELECT COUNT(*) FROM bronze.orders_raw WHERE source = 'web'")),
    ("  of which mobile", 2, q("SELECT COUNT(*) FROM bronze.orders_raw WHERE source = 'mobile'")),
    ("Rows in bronze.customers_raw (tier changes)", 4, q("SELECT COUNT(*) FROM bronze.customers_raw")),
    ("Rows in bronze.fx_raw (FX replies)", 1, q("SELECT COUNT(*) FROM bronze.fx_raw")),
    ("Rows in bronze.pincode_raw (3 on the first run)", 3, q("SELECT COUNT(*) FROM bronze.pincode_raw")),
    ("Rows that pass cleaning (silver.orders_clean)", 6, q("SELECT COUNT(*) FROM silver.orders_clean")),
    ("Rows in quarantine.orders", 2, q("SELECT COUNT(*) FROM quarantine.orders")),
    ("Quarantined order ids", ["ORD-1004", "ORD-1006"], quarantined),
    ("Orders after deduplication (silver.orders_dedup)", 4, q("SELECT COUNT(*) FROM silver.orders_dedup")),
    ("Rows in dim.customers_scd2 (tiers)", 4, q("SELECT COUNT(*) FROM dim.customers_scd2")),
    ("Rows in ref.pincode_geo", 3, q("SELECT COUNT(*) FROM ref.pincode_geo")),
    ("Gold orders (gold.fct_orders)", 4, q("SELECT COUNT(*) FROM gold.fct_orders")),
    ("Gold revenue (INR)", "7785.00", f"{gold_rev:.2f}"),
    ("Gold loyalty points", 126, q("SELECT SUM(loyalty_points) FROM gold.fct_orders")),
    ("Share of rows rejected", "25%", f"{rejected_share:.0%}"),
]
result = pd.DataFrame([(n, e, a, "PASS" if str(e) == str(a) else "FAIL") for n, e, a in rows],
                      columns=["Check", "Expected", "Actual", "Result"])
print(f"{(result.Result == 'PASS').sum()} of {len(result)} checks PASS")
result
```

```
16 of 16 checks PASS
```

Out[17]:

	Check	Expected	Actual	Result
0	Raw order rows in bronze.orders_raw	8	8	PASS
1	of which web	6	6	PASS

2	of which mobile	2	2	PASS
3	Rows in bronze.customers_raw (tier changes)	4	4	PASS
4	Rows in bronze.fx_raw (FX replies)	1	1	PASS
5	Rows in bronze.pincode_raw (3 on the first run)	3	3	PASS
6	Rows that pass cleaning (silver.orders_clean)	6	6	PASS
7	Rows in quarantine.orders	2	2	PASS
8	Quarantined order ids	[ORD-1004, ORD-1006]	[ORD-1004, ORD-1006]	PASS
9	Orders after deduplication (silver.orders_dedup)	4	4	PASS
10	Rows in dim.customers_scd2 (tiers)	4	4	PASS
11	Rows in ref.pincode_geo	3	3	PASS
12	Gold orders (gold.fct_orders)	4	4	PASS
13	Gold revenue (INR)	7785.00	7785.00	PASS
14	Gold loyalty points	126	126	PASS
15	Share of rows rejected	25%	25%	PASS

11. Run it again: nothing should double *(not run yet)*

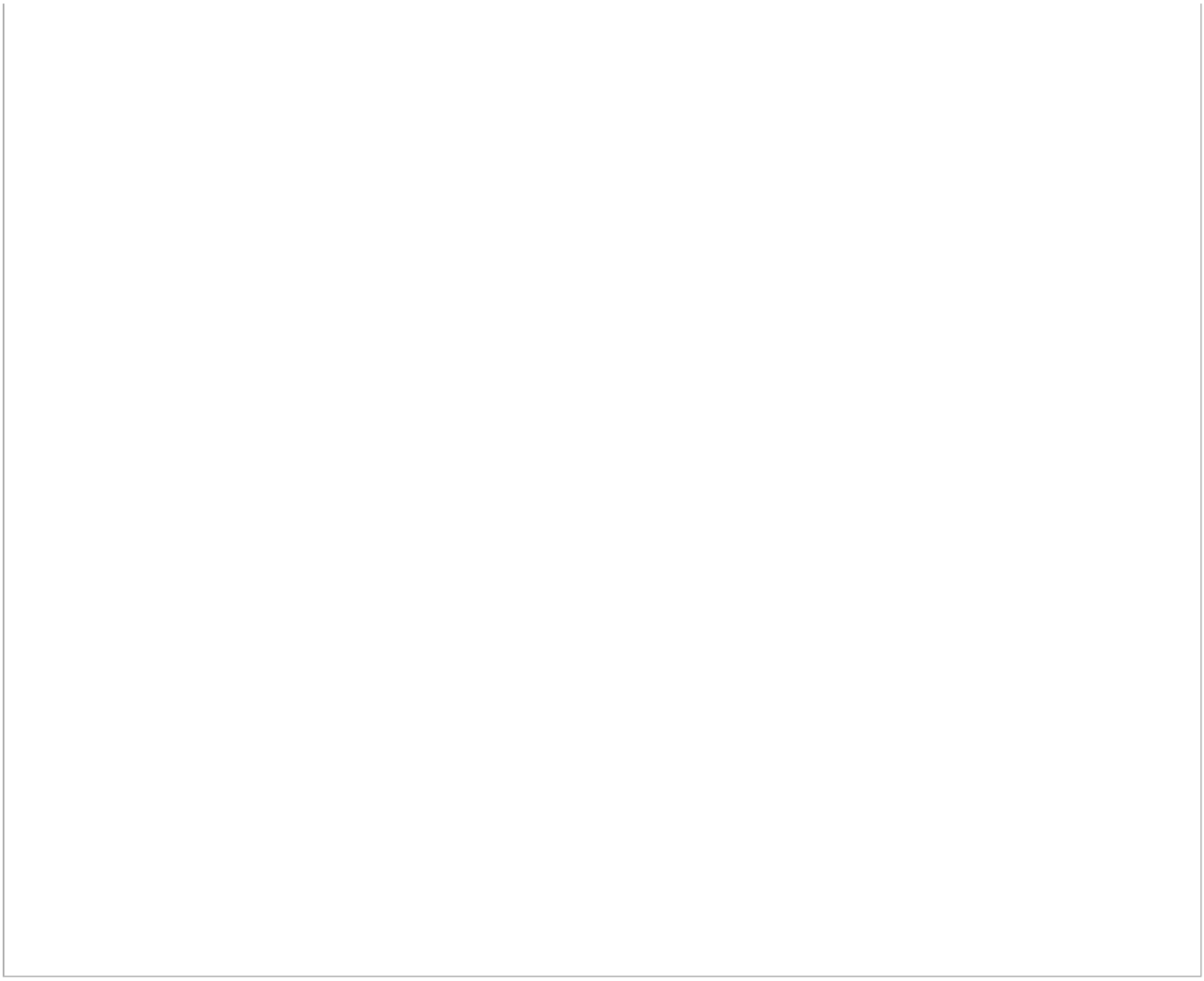
Bronze grows by 8 order rows on every run, because it keeps every copy it is given. Quarantine and gold stay the same. This section is run later (B3/B4), so the B1 evidence above is taken after exactly one run.

In [ ]:
```
fc.run_pipeline()
fc.show("""
SELECT (SELECT COUNT(*) FROM bronze.orders_raw)  AS bronze_rows,
       (SELECT COUNT(*) FROM quarantine.orders)  AS quarantine_rows,
       (SELECT COUNT(*) FROM gold.fct_orders)    AS gold_orders,
       (SELECT SUM(amount_inr) FROM gold.fct_orders) AS revenue_inr,
       (SELECT SUM(loyalty_points) FROM gold.fct_orders) AS points
""")
```

12. Clean up (optional)

Removes everything this notebook created in Snowflake. Uncomment to run.

In [ ]:
```
# fc.query("DROP DATABASE IF EXISTS FRESHCART_DB")
# fc.query("DROP WAREHOUSE IF EXISTS FRESHCART_WH")
fc.conn.close()
```



B3: before / after the currency rule

B3: add a validation rule: `unsupported_currency`

FreshCart only accepts orders in **INR** and **USD**. One line was added inside `ARRAY_CONSTRUCT_COMPACT(...)` in `elt/sql/10_orders_clean.sql` (`SQL_10_ORDERS_CLEAN`). Any other currency (or a missing one) gets the reason `unsupported_currency`. The currency is already upper-cased by then, so `usd` and `inr` still pass.

In [2]:
```
# The reasons block of SQL_10_ORDERS_CLEAN, with the new line (marked B3)
lines = fc.SQL_10_ORDERS_CLEAN.splitlines()
start = next(i for i, l in enumerate(lines) if "ARRAY_CONSTRUCT_COMPACT" in l)
print("\n".join(lines[start:start + 9]))
```

```
 ARRAY_CONSTRUCT_COMPACT(
   IFF(order_id IS NULL, 'missing_order_id', NULL),
   IFF(email IS NULL OR NOT REGEXP_LIKE(email, '^[^@\\s]+@[^@\\s]+\\.[a-z]{2,}$'), 'bad_email', NULL),
   IFF(pincode IS NULL OR NOT REGEXP_LIKE(pincode, '^[1-9][0-9]{5}$'), 'bad_pincode', NULL),
   IFF(currency IS NULL OR currency NOT IN ('INR', 'USD'), 'unsupported_currency', NULL),   -- B3: only INR and USD are accepted
   IFF(order_date IS NULL, 'unparseable_date', NULL),
   IFF(updated_at IS NULL, 'unparseable_updated_at', NULL),
   IFF(amount <= 0, 'non_positive_amount', NULL)          -- NULL amount is allowed (flagged below)
 ) AS reasons
```

Before the re-run

The tables still hold what the **B1 run** produced with the *old* rule. `quarantine.orders` stores the reasons ORD-1006 was given when it was first rejected.

In [3]:
```
import pandas as pd
q = lambda sql: fc.query(sql)[0][0]

def snapshot():
    gold = fc.query("SELECT COUNT(*), SUM(amount_inr), SUM(loyalty_points) FROM gold.fct_orders")[0]
    return {
        "bronze.orders_raw rows": q("SELECT COUNT(*) FROM bronze.orders_raw"),
        "silver.orders_checked rows": q("SELECT COUNT(*) FROM silver.orders_checked"),
        "silver.orders_clean rows": q("SELECT COUNT(*) FROM silver.orders_clean"),
        "silver.orders_dedup rows": q("SELECT COUNT(*) FROM silver.orders_dedup"),
        "quarantine.orders rows": q("SELECT COUNT(*) FROM quarantine.orders"),
        "gold orders": gold[0], "gold revenue_inr": f"{gold[1]:.2f}", "gold loyalty points": gold[2],
    }

before = snapshot()
assert before["bronze.orders_raw rows"] == 8, "expected the B1 state (8 bronze order rows) before the B3 re-run"
print(before)
print("\nORD-1006 in quarantine.orders (BEFORE):")
display(fc.show("SELECT run_id, source, order_id, TO_JSON(reasons) AS reasons FROM quarantine.orders WHERE order_id = 'ORD-1006'"))
print("ORD-1006 in silver.orders_checked (BEFORE):")
display(fc.show("SELECT ingest_seq, source, order_id, currency, TO_JSON(reasons) AS reasons FROM silver.orders_checked WHERE order_id = 'ORD-1006'"))
```

```
{'bronze.orders_raw rows': 8, 'silver.orders_checked rows': 8, 'silver.orders_clean rows': 6, 'silver.orders_dedup rows': 4, 'quarantine.orders rows': 2, 'gold orders': 4, 'gold revenue_inr': '7785.00', 'gold loyalty points': 126}

ORD-1006 in quarantine.orders (BEFORE):
```

	RUN_ID	SOURCE	ORDER_ID	REASONS
0	20261112_040919	web	ORD-1006	["bad_email","bad_pincode"]

```
ORD-1006 in silver.orders_checked (BEFORE):
```

	INGEST_SEQ	SOURCE	ORDER_ID	CURRENCY	REASONS
0	6	web	ORD-1006	EUR	["bad_email","bad_pincode"]

Re-run the pipeline (with the new rule)

In [4]:
```
b3_info = fc.run_pipeline()
```

```
run_id: 20261112_042326
1. EXTRACT: sample data -> raw files
   web: 6 records -> elt\landing\web\web__20261112_042326.json.gz
   mobile: 2 records -> elt\landing\mobile\mobile__20261112_042326.json.gz
   crm: 4 records -> elt\landing\crm\crm__20261112_042326.json.gz
   fx: 1 records -> elt\landing\fx\fx__20261112_042326.json.gz
2. LOAD: raw files -> bronze tables
   6 rows -> bronze.orders_raw
   2 rows -> bronze.orders_raw
   4 rows -> bronze.customers_raw
   1 rows -> bronze.fx_raw
3. TRANSFORM: bronze -> silver -> gold
   new pincodes to look up: []
4. CHECK
   rows gained or lost while enriching    0   OK
   duplicate orders in gold               0   OK
   orders with no customer match          0   OK
   orders with an amount but no FX rate   0   OK
   orders with no city                    0   OK
   share of rows rejected                 25%
GOLD: 4 orders, revenue ₹7785.00, 126 loyalty points
```

After the re-run

In [5]:
```
after = snapshot()
comparison = pd.DataFrame({"before": before, "after": after})
comparison["changed?"] = ["yes" if str(b) != str(a) else "no" for b, a in zip(comparison["before"], comparison["after"])]
comparison
```

Out[5]:

	before	after	changed?
bronze.orders_raw rows	8	16	yes
silver.orders_checked rows	8	16	yes
silver.orders_clean rows	6	12	yes
silver.orders_dedup rows	4	4	no
quarantine.orders rows	2	2	no
gold orders	4	4	no
gold revenue_inr	7785.00	7785.00	no

gold loyalty points	126	126	no

In [6]:
```
print("ORD-1006 in silver.orders_checked (AFTER; rebuilt by the re-run from all bronze rows):")
display(fc.show("SELECT ingest_seq, source, order_id, currency, TO_JSON(reasons) AS reasons FROM silver.orders_checked WHERE order_id = 'ORD-1006' ORDER BY ingest_s
print("ORD-1006 in quarantine.orders (AFTER):")
display(fc.show("SELECT run_id, source, order_id, TO_JSON(reasons) AS reasons FROM quarantine.orders WHERE order_id = 'ORD-1006'"))
```

```
ORD-1006 in silver.orders_checked (AFTER; rebuilt by the re-run from all bronze rows):
```

	INGEST_SEQ	SOURCE	ORDER_ID	CURRENCY	REASONS
0	6	web	ORD-1006	EUR	["bad_email","bad_pincode","unsupported_curren…
1	14	web	ORD-1006	EUR	["bad_email","bad_pincode","unsupported_curren…

```
ORD-1006 in quarantine.orders (AFTER):
```

	RUN_ID	SOURCE	ORDER_ID	REASONS
0	20261112_040919	web	ORD-1006	["bad_email","bad_pincode"]

Is only the right currency flagged?

In [7]:
```
fc.show("""
SELECT currency, COUNT(*) AS rows_in_orders_checked,
       COUNT_IF(ARRAY_CONTAINS('unsupported_currency'::VARIANT, reasons)) AS flagged_unsupported_currency
FROM silver.orders_checked GROUP BY currency ORDER BY currency
""")
```

Out[7]:

	CURRENCY	ROWS_IN_ORDERS_CHECKED	FLAGGED_UNSUPPORTED_CURRENCY
0	EUR	2	2
1	INR	10	0
2	USD	4	0

Did the re-run add a second quarantine row?

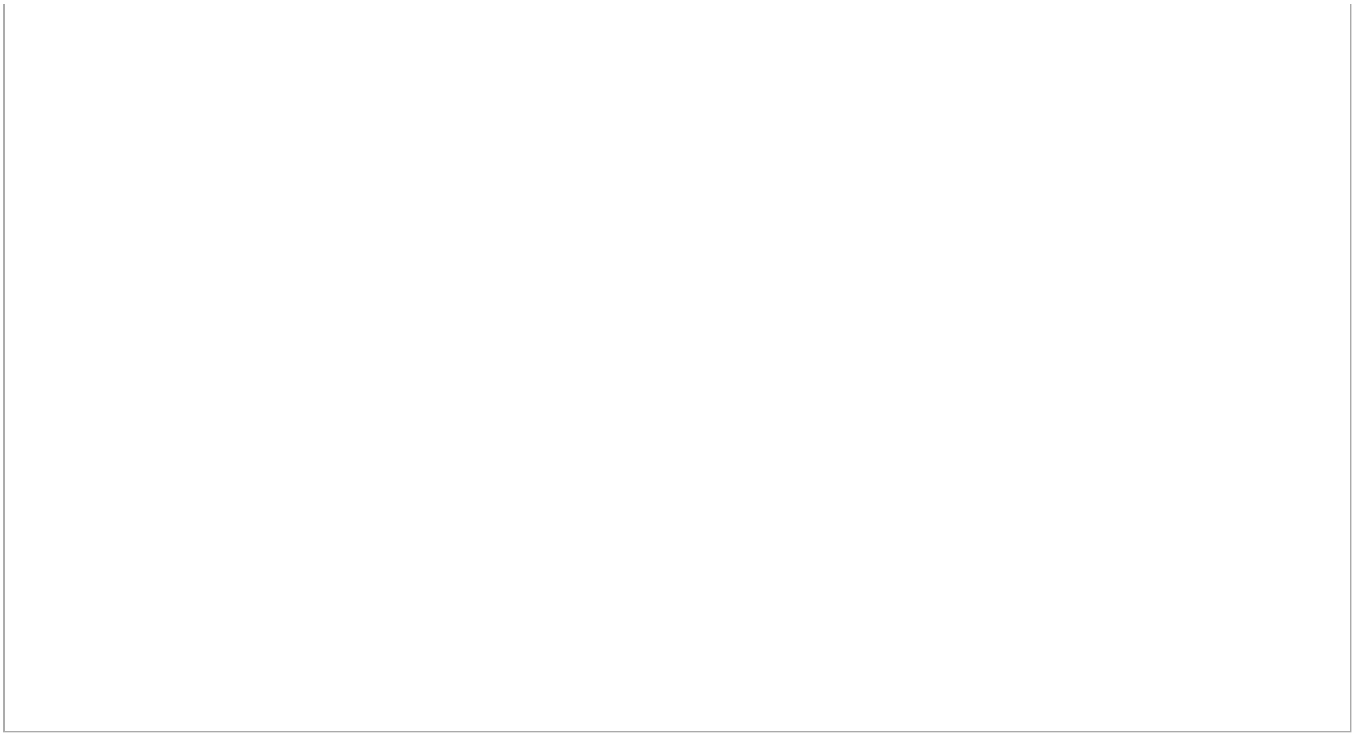
Bronze now holds **two copies** of every raw order (one per run), so `silver.orders_checked` has two copies of each rejected row, but `quarantine.orders` should still have one row per rejected record. The `NOT EXISTS (...)` in `SQL_10_ORDERS_CLEAN` skips a rejected row when `quarantine.orders` already has the same `source`, `order_id` and `updated_at` (it does not look at the reasons).

In [8]:
```
print("copies of each rejected record in silver.orders_checked vs quarantine.orders:")
fc.show("""
SELECT c.source, c.order_id, c.updated_at,
       COUNT(*) AS copies_in_orders_checked,
       (SELECT COUNT(*) FROM quarantine.orders q
         WHERE q.source = c.source AND q.order_id = c.order_id
           AND EQUAL_NULL(q.row_data:UPDATED_AT::TIMESTAMP_NTZ, c.updated_at)) AS rows_in_quarantine
FROM silver.orders_checked c
WHERE ARRAY_SIZE(c.reasons) > 0
GROUP BY c.source, c.order_id, c.updated_at ORDER BY c.order_id
""")
```

```
copies of each rejected record in silver.orders_checked vs quarantine.orders:
```

Out[8]:

	SOURCE	ORDER_ID	UPDATED_AT	COPIES_IN_ORDERS_CHECKED	ROWS_IN_QUARANTINE
0	web	ORD-1004	2026-09-03 12:00:00	2	1
1	web	ORD-1006	2026-09-04 08:30:00	2	1



B4: five outcomes, idempotence, bronze growth

B4: customer tier change (SCD2) and idempotency

New CRM event added to `MOCK_CRM_TIER_EVENTS` in `elt/mock_data.py` : **CUST-02, arjun.k@yahoo.in, Gold, effective_from 2026-08-15**. The predictions below were written and committed **before** this section was run (`report/answers.md` , "PRE-RUN PREDICTIONS").

In [2]:
```
import pandas as pd
q = lambda sql: fc.query(sql)[0][0]
print("events in MOCK_CRM_TIER_EVENTS:", len(fc.MOCK_CRM_TIER_EVENTS))
pd.DataFrame(fc.MOCK_CRM_TIER_EVENTS)
```

```
events in MOCK_CRM_TIER_EVENTS: 5
```

Out[2]:

	cust_id	email	tier	effective_from
0	CUST-01	priya.r@gmail.com	Silver	2023-04-10
1	CUST-01	priya.r@gmail.com	Gold	2026-09-03
2	CUST-02	arjun.k@yahoo.in	Silver	2024-01-15
3	CUST-03	meera@outlook.com	Platinum	2022-11-01
4	CUST-02	arjun.k@yahoo.in	Gold	2026-08-15

Before the run: gold rows with `published_at` and `row_hash`

In [3]:
```
gold_before = fc.show("SELECT order_id, tier, multiplier, loyalty_points, row_hash, published_at FROM gold.fct_orders ORDER BY order_id")
bronze_before = {t: q(f"SELECT COUNT(*) FROM bronze.{t}") for t in ["orders_raw", "customers_raw", "fx_raw", "pincode_raw"]}
print("CUST-02 rows in dim.customers_scd2 BEFORE:", q("SELECT COUNT(*) FROM dim.customers_scd2 WHERE cust_id = 'CUST-02'"))
print("bronze row counts BEFORE:", bronze_before)
gold_before
```

```
CUST-02 rows in dim.customers_scd2 BEFORE: 1
bronze row counts BEFORE: {'orders_raw': 16, 'customers_raw': 8, 'fx_raw': 2, 'pincode_raw': 3}
```

Out[3]:

	ORDER_ID	TIER	MULTIPLIER	LOYALTY_POINTS	ROW_HASH	PUBLISHED_AT
0	ORD-1001	Silver	1	12	4284627330879666653	2026-11-11 20:09:29.606000-08:00
1	ORD-1002	Silver	1	35	-7698738864412791922	2026-11-11 20:09:29.606000-08:00
2	ORD-1003	Platinum	3	63	-1632326326823866187	2026-11-11 20:09:29.606000-08:00
3	ORD-1005	Gold	2	16	-231880702048109571	2026-11-11 20:09:29.606000-08:00

Run the pipeline once (full run: extract, load, transform, check)

In [4]:
```
b4_info = fc.run_pipeline()
```

```
run_id: 20261112_043003
1. EXTRACT: sample data -> raw files
   web: 6 records -> elt\landing\web\web__20261112_043003.json.gz
   mobile: 2 records -> elt\landing\mobile\mobile__20261112_043003.json.gz
   crm: 5 records -> elt\landing\crm\crm__20261112_043003.json.gz
   fx: 1 records -> elt\landing\fx\fx__20261112_043003.json.gz
2. LOAD: raw files -> bronze tables
   6 rows -> bronze.orders_raw
   2 rows -> bronze.orders_raw
   5 rows -> bronze.customers_raw
   1 rows -> bronze.fx_raw
3. TRANSFORM: bronze -> silver -> gold
   new pincodes to look up: []
4. CHECK
   rows gained or lost while enriching    0   OK
   duplicate orders in gold               0   OK
   orders with no customer match          0   OK
   orders with an amount but no FX rate   0   OK
   orders with no city                    0   OK
   share of rows rejected                 25%
GOLD: 4 orders, revenue ₹7785.00, 161 loyalty points
```

The five outcomes: committed prediction vs actual

In [2]:
```
import pandas as pd
q = lambda sql: fc.query(sql)[0][0]

# The B4 pipeline run above has run_id 20261112_043003 (= 04:30:03 UTC). Snowflake Time Travel gives the tables exactly as
# they were just before it, so the "before" side of every comparison is read from Snowflake itself, not retyped.
BEFORE_RUN = "2026-11-12 04:30:00 +0000"
at = f"AT(TIMESTAMP => '{BEFORE_RUN}'::TIMESTAMP_LTZ)"
gold_before = fc.show(f"SELECT order_id, tier, multiplier, loyalty_points, row_hash, published_at FROM gold.fct_orders {at} ORDER BY order_id")
bronze_before = {t: q(f"SELECT COUNT(*) FROM bronze.{t} {at}") for t in ["orders_raw", "customers_raw", "fx_raw", "pincode_raw"]}

gold_after = fc.show("SELECT order_id, tier, multiplier, loyalty_points, row_hash, published_at FROM gold.fct_orders ORDER BY order_id")
cmp_ = gold_before.merge(gold_after, on="ORDER_ID", suffixes=("_before", "_after"))
published_changed = int((cmp_["PUBLISHED_AT_before"] != cmp_["PUBLISHED_AT_after"]).sum())
hash_changed = int((cmp_["ROW_HASH_before"] != cmp_["ROW_HASH_after"]).sum())
ord1002 = gold_after[gold_after.ORDER_ID == "ORD-1002"].iloc[0]
actual = {
    "CUST-02 rows in dim.customers_scd2": q("SELECT COUNT(*) FROM dim.customers_scd2 WHERE cust_id = 'CUST-02'"),
    "Tier used for ORD-1002": ord1002.TIER,
    "Loyalty points for ORD-1002": int(ord1002.LOYALTY_POINTS),
    "Total loyalty points in gold": int(q("SELECT SUM(loyalty_points) FROM gold.fct_orders")),
    "Gold rows whose published_at changed": published_changed,
}
predicted = {  # copied from the committed PRE-RUN PREDICTIONS in report/answers.md (commit 72e290d)
    "CUST-02 rows in dim.customers_scd2": 2,
    "Tier used for ORD-1002": "Gold",
    "Loyalty points for ORD-1002": 70,
    "Total loyalty points in gold": 161,
    "Gold rows whose published_at changed": 1,
}
result = pd.DataFrame({"prediction": predicted, "actual": actual})
result["match?"] = ["YES" if str(p) == str(a) else "NO" for p, a in zip(result["prediction"], result["actual"])]
print("gold BEFORE the B4 run (Time Travel):")
display(gold_before)
print(f"{(result['match?'] == 'YES').sum()} of {len(result)} predictions match; rows with changed row_hash: {hash_changed}")
result
```

```
gold BEFORE the B4 run (Time Travel):
```

	ORDER_ID	TIER	MULTIPLIER	LOYALTY_POINTS	ROW_HASH	PUBLISHED_AT
0	ORD-1001	Silver	1	12	4284627330879666653	2026-11-11 20:09:29.606000-08:00
1	ORD-1002	Silver	1	35	-7698738864412791922	2026-11-11 20:09:29.606000-08:00
2	ORD-1003	Platinum	3	63	-1632326326823866187	2026-11-11 20:09:29.606000-08:00
3	ORD-1005	Gold	2	16	-231880702048109571	2026-11-11 20:09:29.606000-08:00

```
5 of 5 predictions match; rows with changed row_hash: 1
```

Out[2]:

	prediction	actual	match?
CUST-02 rows in dim.customers_scd2	2	2	YES
Tier used for ORD-1002	Gold	Gold	YES
Loyalty points for ORD-1002	70	70	YES
Total loyalty points in gold	161	161	YES
Gold rows whose published_at changed	1	1	YES

In [3]:
```
print("CUST-02 in dim.customers_scd2 (AFTER):")
display(fc.show("SELECT cust_id, tier, valid_from, valid_to, is_current FROM dim.customers_scd2 WHERE cust_id = 'CUST-02' ORDER BY valid_from"))
print("Gold rows: before vs after (only ORD-1002 should differ):")
cmp_
```

```
CUST-02 in dim.customers_scd2 (AFTER):
```

	CUST_ID	TIER	VALID_FROM	VALID_TO	IS_CURRENT
0	CUST-02	Silver	2024-01-15	2026-08-14	False
1	CUST-02	Gold	2026-08-15	9999-12-31	True

```
Gold rows: before vs after (only ORD-1002 should differ):
```

Out[3]:

	ORDER_ID	TIER_before	MULTIPLIER_before	LOYALTY_POINTS_before	ROW_HASH_before	PUBLISHED_AT_before	TIER_after	MULTIPLIER_after	LOYALTY_POINTS_after	R
0	ORD-1001	Silver	1	12	4284627330879666653	2026-11-11 20:09:29.606000-08:00	Silver	1	12	42846
1	ORD-1002	Silver	1	35	-7698738864412791922	2026-11-11 20:09:29.606000-08:00	Gold	2	70	33922
2	ORD-1003	Platinum	3	63	-1632326326823866187	2026-11-11 20:09:29.606000-08:00	Platinum	3	63	-16323
3	ORD-1005	Gold	2	16	-231880702048109571	2026-11-11 20:09:29.606000-08:00	Gold	2	16	-2318

Idempotence: run the transformation a second time

Only the transformation is repeated (`SQL_30_CUSTOMERS_SCD2` rebuilds the dimension, `SQL_40_GOLD` rebuilds `silver.orders_enriched` and runs the `MERGE`). Nothing new is loaded. The `MERGE` updates a row (and refreshes `published_at`) only when `row_hash` changed, so every `published_at` must stay as it was after the first B4 run.

In [4]:
```
gold_mid = fc.show("SELECT order_id, row_hash, published_at FROM gold.fct_orders ORDER BY order_id")
fc.run_sql(fc.SQL_30_CUSTOMERS_SCD2)
fc.run_sql(fc.SQL_40_GOLD)
gold_again = fc.show("SELECT order_id, row_hash, published_at FROM gold.fct_orders ORDER BY order_id")
idem = gold_mid.merge(gold_again, on="ORDER_ID", suffixes=("_run1", "_run2"))
idem["published_at_kept?"] = idem["PUBLISHED_AT_run1"] == idem["PUBLISHED_AT_run2"]
idem["row_hash_same?"] = idem["ROW_HASH_run1"] == idem["ROW_HASH_run2"]
```

```
print("rows whose published_at changed in the second pass:", int((~idem["published_at_kept?"]).sum()))
print("gold totals after the second pass:", fc.query("SELECT COUNT(*), SUM(amount_inr), SUM(loyalty_points) FROM gold.fct_orders")[0])
idem
```

```
rows whose published_at changed in the second pass: 0
gold totals after the second pass: (4, Decimal('7785.00'), 161)
```

Out[4]:

	ORDER_ID	ROW_HASH_run1	PUBLISHED_AT_run1	ROW_HASH_run2	PUBLISHED_AT_run2	published_at_kept?	row_hash_same?
0	ORD-1001	4284627330879666653	2026-11-11 20:09:29.606000-08:00	4284627330879666653	2026-11-11 20:09:29.606000-08:00	True	True
1	ORD-1002	3392257107350681567	2026-11-11 20:30:13.292000-08:00	3392257107350681567	2026-11-11 20:30:13.292000-08:00	True	True
2	ORD-1003	-16323263268238661 87	2026-11-11 20:09:29.606000-08:00	-16323263268238661 87	2026-11-11 20:09:29.606000-08:00	True	True
3	ORD-1005	-231880702048109571	2026-11-11 20:09:29.606000-08:00	-231880702048109571	2026-11-11 20:09:29.606000-08:00	True	True

Why does bronze grow by 8 order rows on every run?

Every run extracts the same 8 raw orders again, writes them to **new landing files** (the file name contains the new `run_id`), `PUT`s them to a new stage folder and `COPY INTO` appends them. Bronze keeps every copy it is given (it has no deduplication; that is silver's job), so `bronze.orders_raw` grows by 6 web + 2 mobile = 8 rows per run, while silver (`orders_dedup`) and gold stay the same size.

In [5]:
```
print("bronze.orders_raw rows per run (each full pipeline run adds one group of 8):")
display(fc.show("SELECT run_id, COUNT(*) AS order_rows, COUNT_IF(source = 'web') AS web, COUNT_IF(source = 'mobile') AS mobile, COUNT(DISTINCT src_file) AS landing_
bronze_after = {t: q(f"SELECT COUNT(*) FROM bronze.{t}") for t in ["orders_raw", "customers_raw", "fx_raw", "pincode_raw"]}
growth = pd.DataFrame({"before B4 run": bronze_before, "after B4 run": bronze_after})
growth["added by this run"] = growth["after B4 run"] - growth["before B4 run"]
print("this run added 8 order rows, 5 CRM rows (the 4 old events + the new one), 1 FX reply, and no pincodes (all already known):")
growth
```

```
bronze.orders_raw rows per run (each full pipeline run adds one group of 8):
```

	RUN_ID	ORDER_ROWS	WEB	MOBILE	LANDING_FILES
0	20261112_040919	8	6	2	2
1	20261112_042326	8	6	2	2
2	20261112_043003	8	6	2	2

```
this run added 8 order rows, 5 CRM rows (the 4 old events + the new one), 1 FX reply, and no pincodes (all already known):
```

Out[5]:

	before B4 run	after B4 run	added by this run
orders_raw	16	24	8
customers_raw	8	13	5
fx_raw	2	3	1
pincode_raw	3	3	0



B5: PROBLEM output, cleanup and recovery

B5: a sixth quality check: orders dated in the future

`quality_checks()` (in `elt/freshcart.py`) now has a sixth check that should also be **0**:

```
SELECT COUNT_IF(order_date > CURRENT_DATE()) FROM silver.orders_dedup      -- "orders dated in the future"
```

Plan: (1) show it passes on the real data, (2) make it fail on purpose with **one clearly marked mock order** (`TEST-B5-FUTURE-DATE-001`), (3) clean up exactly that test data, (4) show the check is back to 0.

In [2]:
```
import inspect
import pandas as pd
q = lambda sql: fc.query(sql)[0][0]

print("the new check inside quality_checks():")
print("\n".join(l for l in inspect.getsource(fc.quality_checks).splitlines() if "B5" in l or "in the future" in l or "CURRENT_DATE" in l))
print("\nsession CURRENT_DATE():", q("SELECT CURRENT_DATE()"))
print("\nCONTROL: quality_checks() on the real data (the new check must be 0 / OK):")
fc.quality_checks()
```

```
the new check inside quality_checks():
        # B5: the sixth check. An order cannot have happened in the future.
        "orders dated in the future":
            "SELECT COUNT_IF(order_date > CURRENT_DATE()) FROM silver.orders_dedup",
session CURRENT_DATE(): 2026-11-11

CONTROL: quality_checks() on the real data (the new check must be 0 / OK):
   rows gained or lost while enriching    0   OK
   duplicate orders in gold               0   OK
   orders with no customer match          0   OK
   orders with an amount but no FX rate   0   OK
   orders with no city                    0   OK
   orders dated in the future             0   OK
   share of rows rejected                 25%
```

The mock order

It is a *web-format* order with a future `order_date`. It is built so that **only the new check fails**: it uses an existing customer email and an existing pincode (so customer match and city are fine) and has **no amount** (a missing amount is allowed, and there is no FX rate for a future date, so the FX check stays quiet). Everything about it is marked: the order id `TEST-B5-FUTURE-DATE-001` and the run id ending in `_b5test`. A baseline of every table is taken first, so cleanup can be checked against it.

In [3]:
```
from datetime import datetime, timezone

TEST_ORDER_ID = "TEST-B5-FUTURE-DATE-001"
TEST_RUN_ID = datetime.now(timezone.utc).strftime("%Y%m%d_%H%M%S") + "_b5test"
test_order = {"order_id": TEST_ORDER_ID, "cust_email": "priya.r@gmail.com", "cust_name": "B5 test order",
              "order_date": "2026-12-31", "amount": None, "currency": "INR", "pincode": "600041",
              "updated_at": "2026-11-12 09:00:00"}

tables = ["bronze.orders_raw", "bronze.customers_raw", "bronze.fx_raw", "bronze.pincode_raw", "silver.orders_checked",
          "silver.orders_clean", "silver.orders_dedup", "silver.orders_enriched", "dim.customers_scd2", "ref.fx_daily",
          "ref.pincode_geo", "quarantine.orders", "gold.fct_orders"]
baseline = {t: q(f"SELECT COUNT(*) FROM {t}") for t in tables}
baseline_gold = fc.show("SELECT order_id, tier, amount_inr, loyalty_points, row_hash, published_at FROM gold.fct_orders ORDER BY order_id")

traces = {"bronze": q(f"SELECT COUNT(*) FROM bronze.orders_raw WHERE raw:order_id::STRING = '{TEST_ORDER_ID}' OR run_id = '{TEST_RUN_ID}'"),
          "gold": q(f"SELECT COUNT(*) FROM gold.fct_orders WHERE order_id = '{TEST_ORDER_ID}'"),
          "quarantine": q(f"SELECT COUNT(*) FROM quarantine.orders WHERE order_id = '{TEST_ORDER_ID}'")}
assert set(traces.values()) == {0}, f"the marker already exists somewhere: {traces}"
print("test order:", test_order)
print("run id    :", TEST_RUN_ID)
print("marker not present anywhere yet:", traces)
print("baseline row counts:", baseline)
baseline_gold
```

```
test order: {'order_id': 'TEST-B5-FUTURE-DATE-001', 'cust_email': 'priya.r@gmail.com', 'cust_name': 'B5 test order', 'order_date': '2026-12-31', 'amount': None, 'cur
rency': 'INR', 'pincode': '600041', 'updated_at': '2026-11-12 09:00:00'}
run id    : 20261112_043630_b5test
marker not present anywhere yet: {'bronze': 0, 'gold': 0, 'quarantine': 0}
baseline row counts: {'bronze.orders_raw': 24, 'bronze.customers_raw': 13, 'bronze.fx_raw': 3, 'bronze.pincode_raw': 3, 'silver.orders_checked': 24, 'silver.orders_c
lean': 18, 'silver.orders_dedup': 4, 'silver.orders_enriched': 4, 'dim.customers_scd2': 5, 'ref.fx_daily': 144, 'ref.pincode_geo': 3, 'quarantine.orders': 2, 'gold.f
ct_orders': 4}
```

Out[3]:

	ORDER_ID	TIER	AMOUNT_INR	LOYALTY_POINTS	ROW_HASH	PUBLISHED_AT
0	ORD-1001	Silver	1250.00	12	4284627330879666653	2026-11-11 20:09:29.606000-08:00
1	ORD-1002	Gold	3536.00	70	3392257107350681567	2026-11-11 20:30:13.292000-08:00
2	ORD-1003	Platinum	2100.00	63	-1632326326823866187	2026-11-11 20:09:29.606000-08:00
3	ORD-1005	Gold	899.00	16	-231880702048109571	2026-11-11 20:09:29.606000-08:00

Load the mock order, rebuild silver and gold, run the checks (expect a PROBLEM)

In [4]:
```
print("1. EXTRACT + 2. LOAD: only the mock order, in the web format")
test_file = fc.write_landing("web", [test_order], TEST_RUN_ID)
fc.load_file(test_file, "bronze.orders_raw", f"orders_web/{TEST_RUN_ID}")
print("3. TRANSFORM: bronze -> silver -> gold")
fc.transform(TEST_RUN_ID)
print("4. CHECK")
fc.quality_checks()
```

```
1. EXTRACT + 2. LOAD: only the mock order, in the web format
   web: 1 records -> elt\landing\web\web__20261112_043630_b5test.json.gz
   1 rows -> bronze.orders_raw
3. TRANSFORM: bronze -> silver -> gold
   new pincodes to look up: []
4. CHECK
   rows gained or lost while enriching    0   OK
   duplicate orders in gold               0   OK
   orders with no customer match          0   OK
   orders with an amount but no FX rate   0   OK
   orders with no city                    0   OK
   orders dated in the future             1   PROBLEM
   share of rows rejected                 24%
```

In [5]:
```
print("the offending row in silver.orders_dedup (order_date > CURRENT_DATE()):")
display(fc.show("SELECT order_id, order_date, email, amount, currency, pincode FROM silver.orders_dedup WHERE order_date > CURRENT_DATE()"))
print("the mock order has also reached gold.fct_orders (gold's MERGE never deletes):")
display(fc.show("SELECT order_id, order_date, tier, amount_inr, loyalty_points, city FROM gold.fct_orders ORDER BY order_id"))
```

```
print("gold totals now:", fc.query("SELECT COUNT(*), SUM(amount_inr), SUM(loyalty_points) FROM gold.fct_orders")[0])
```

```
the offending row in silver.orders_dedup (order_date > CURRENT_DATE()):
```

	ORDER_ID	ORDER_DATE	EMAIL	AMOUNT	CURRENCY	PINCODE
0	TEST-B5-FUTURE-DATE-001	2026-12-31	priya.r@gmail.com	None	INR	600041

```
the mock order has also reached gold.fct_orders (gold's MERGE never deletes):
```

	ORDER_ID	ORDER_DATE	TIER	AMOUNT_INR	LOYALTY_POINTS	CITY
0	ORD-1001	2026-09-01	Silver	1250.00	12.0	Chennai
1	ORD-1002	2026-09-01	Gold	3536.00	70.0	Bengaluru
2	ORD-1003	2026-09-02	Platinum	2100.00	63.0	Mumbai
3	ORD-1005	2026-09-03	Gold	899.00	16.0	Chennai
4	TEST-B5-FUTURE-DATE-001	2026-12-31	Gold	None	NaN	Chennai

```
gold totals now: (5, Decimal('7785.00'), 161)
```

Cleanup: look first, delete only what is proven to be the test data

Bronze is append-only and gold's `MERGE` never deletes, so removing the order from the Python list would not remove it. From the SQL (`elt/sql/`): `silver.*` , `dim.*` and `ref.fx_daily` are `CREATE OR REPLACE` (rebuilt from bronze on every run, nothing to delete), `quarantine.orders` only inserts (the mock order is valid, so it must have 0 rows there), and `PUT` leaves the file in the stage. So the test data can only be in: `bronze.orders_raw` , `gold.fct_orders` and the stage folder of the test run (plus the local landing file). The cell below counts exactly what matches the markers and **stops (assert) before deleting anything** if the numbers are not exactly as expected.

In [6]:
```
marker = f"raw:order_id::STRING = '{TEST_ORDER_ID}' AND run_id = '{TEST_RUN_ID}'"
staged = fc.query(f"LIST @bronze.landing/orders_web/{TEST_RUN_ID}/")
found = {
    "bronze.orders_raw: order id AND test run id": q(f"SELECT COUNT(*) FROM bronze.orders_raw WHERE {marker}"),
    "bronze.orders_raw: order id (any run)": q(f"SELECT COUNT(*) FROM bronze.orders_raw WHERE raw:order_id::STRING = '{TEST_ORDER_ID}'"),
    "bronze.orders_raw: test run id (any order)": q(f"SELECT COUNT(*) FROM bronze.orders_raw WHERE run_id = '{TEST_RUN_ID}'"),
    "bronze.orders_raw: all OTHER rows (kept)": q(f"SELECT COUNT(*) FROM bronze.orders_raw WHERE run_id <> '{TEST_RUN_ID}'"),
    "gold.fct_orders: this order id": q(f"SELECT COUNT(*) FROM gold.fct_orders WHERE order_id = '{TEST_ORDER_ID}'"),
```

```
    "gold.fct_orders: all OTHER rows (kept)": q(f"SELECT COUNT(*) FROM gold.fct_orders WHERE order_id <> '{TEST_ORDER_ID}'"),
    "quarantine.orders: this order id": q(f"SELECT COUNT(*) FROM quarantine.orders WHERE order_id = '{TEST_ORDER_ID}'"),
    "staged files in this run's stage folder": len(staged),
}
expected = {
    "bronze.orders_raw: order id AND test run id": 1, "bronze.orders_raw: order id (any run)": 1,
    "bronze.orders_raw: test run id (any order)": 1,
    "bronze.orders_raw: all OTHER rows (kept)": baseline["bronze.orders_raw"],
    "gold.fct_orders: this order id": 1, "gold.fct_orders: all OTHER rows (kept)": baseline["gold.fct_orders"],
    "quarantine.orders: this order id": 0, "staged files in this run's stage folder": 1,
}
print(pd.DataFrame({"found": found, "expected": expected}))
assert found == expected, "STOP: the markers match something unexpected; nothing has been deleted"
print("\nrows that WILL be deleted:")
display(fc.show(f"SELECT ingest_seq, source, run_id, TO_JSON(raw) AS raw_json FROM bronze.orders_raw WHERE {marker}"))
display(fc.show(f"SELECT order_id, order_date, tier, amount_inr FROM gold.fct_orders WHERE order_id = '{TEST_ORDER_ID}'"))
print("staged file:", [r[0] for r in staged])
```

```
                                          found  expected
bronze.orders_raw: order id AND test run id      1         1
bronze.orders_raw: order id (any run)            1         1
bronze.orders_raw: test run id (any order)       1         1
bronze.orders_raw: all OTHER rows (kept)        24        24
gold.fct_orders: this order id                   1         1
gold.fct_orders: all OTHER rows (kept)           4         4
quarantine.orders: this order id                 0         0
staged files in this run's stage folder          1         1

rows that WILL be deleted:
```

	INGEST_SEQ	SOURCE	RUN_ID	RAW_JSON
0	25	web	20261112_043630_b5test	{"amount":null,"currency":"INR","cust_email":"...

	ORDER_ID	ORDER_DATE	TIER	AMOUNT_INR
0	TEST-B5-FUTURE-DATE-001	2026-12-31	Gold	None

```
staged file: ['landing/orders_web/20261112_043630_b5test/web__20261112_043630_b5test.json.gz']
```

In [7]:
```
import os
n_bronze = fc.query(f"DELETE FROM bronze.orders_raw WHERE {marker}")[0][0]
n_gold = fc.query(f"DELETE FROM gold.fct_orders WHERE order_id = '{TEST_ORDER_ID}'")[0][0]
removed = fc.query(f"REMOVE @bronze.landing/orders_web/{TEST_RUN_ID}/")
assert TEST_RUN_ID in str(test_file) and "landing" in str(test_file)
os.remove(test_file)
print(f"deleted from bronze.orders_raw: {n_bronze} row | deleted from gold.fct_orders: {n_gold} row")
print("removed from the stage:", [r[:2] for r in removed])
print("local landing file removed:", not os.path.exists(test_file))
assert n_bronze == 1 and n_gold == 1
```

```
deleted from bronze.orders_raw: 1 row | deleted from gold.fct_orders: 1 row
removed from the stage: [('landing/orders_web/20261112_043630_b5test/web__20261112_043630_b5test.json.gz', 'removed')]
local landing file removed: True
```

Recovery: rebuild from the cleaned bronze and re-run the checks

`silver.orders_dedup` still contains the mock order until the transformation runs again, so the stage-3 SQL is re-run (no extract, no load). Then every table must be back to the baseline taken before the mock order existed.

In [8]:
```
print("3. TRANSFORM again (from the cleaned bronze)")
fc.transform("b5_cleanup")
print("4. CHECK")
fc.quality_checks()
```

```
3. TRANSFORM again (from the cleaned bronze)
   new pincodes to look up: []
4. CHECK
   rows gained or lost while enriching    0   OK
   duplicate orders in gold               0   OK
   orders with no customer match          0   OK
   orders with an amount but no FX rate   0   OK
   orders with no city                    0   OK
   orders dated in the future             0   OK
   share of rows rejected                 25%
```

In [9]:
```
after = {t: q(f"SELECT COUNT(*) FROM {t}") for t in tables}
gold_after = fc.show("SELECT order_id, tier, amount_inr, loyalty_points, row_hash, published_at FROM gold.fct_orders ORDER BY order_id")
leftovers = {
    "bronze.orders_raw rows with the marker": q(f"SELECT COUNT(*) FROM bronze.orders_raw WHERE raw:order_id::STRING = '{TEST_ORDER_ID}' OR run_id = '{TEST_RUN_ID}'"
    "gold.fct_orders rows with the marker": q(f"SELECT COUNT(*) FROM gold.fct_orders WHERE order_id = '{TEST_ORDER_ID}'"),
    "quarantine.orders rows with the marker": q(f"SELECT COUNT(*) FROM quarantine.orders WHERE order_id = '{TEST_ORDER_ID}'"),
    "silver.orders_dedup future-dated rows": q("SELECT COUNT_IF(order_date > CURRENT_DATE()) FROM silver.orders_dedup"),
    "staged files for the test run": len(fc.query(f"LIST @bronze.landing/orders_web/{TEST_RUN_ID}/")),
}
rows = [(t, baseline[t], after[t], "PASS" if baseline[t] == after[t] else "FAIL") for t in tables]
rows += [("gold rows identical to baseline (incl. row_hash and published_at)", "yes", "yes" if baseline_gold.equals(gold_after) else "NO",
          "PASS" if baseline_gold.equals(gold_after) else "FAIL")]
rows += [(k, 0, v, "PASS" if v == 0 else "FAIL") for k, v in leftovers.items()]
result = pd.DataFrame(rows, columns=["Check", "Baseline / expected", "Now", "Result"])
print(f"{(result.Result == 'PASS').sum()} of {len(result)} cleanup checks PASS")
result
```

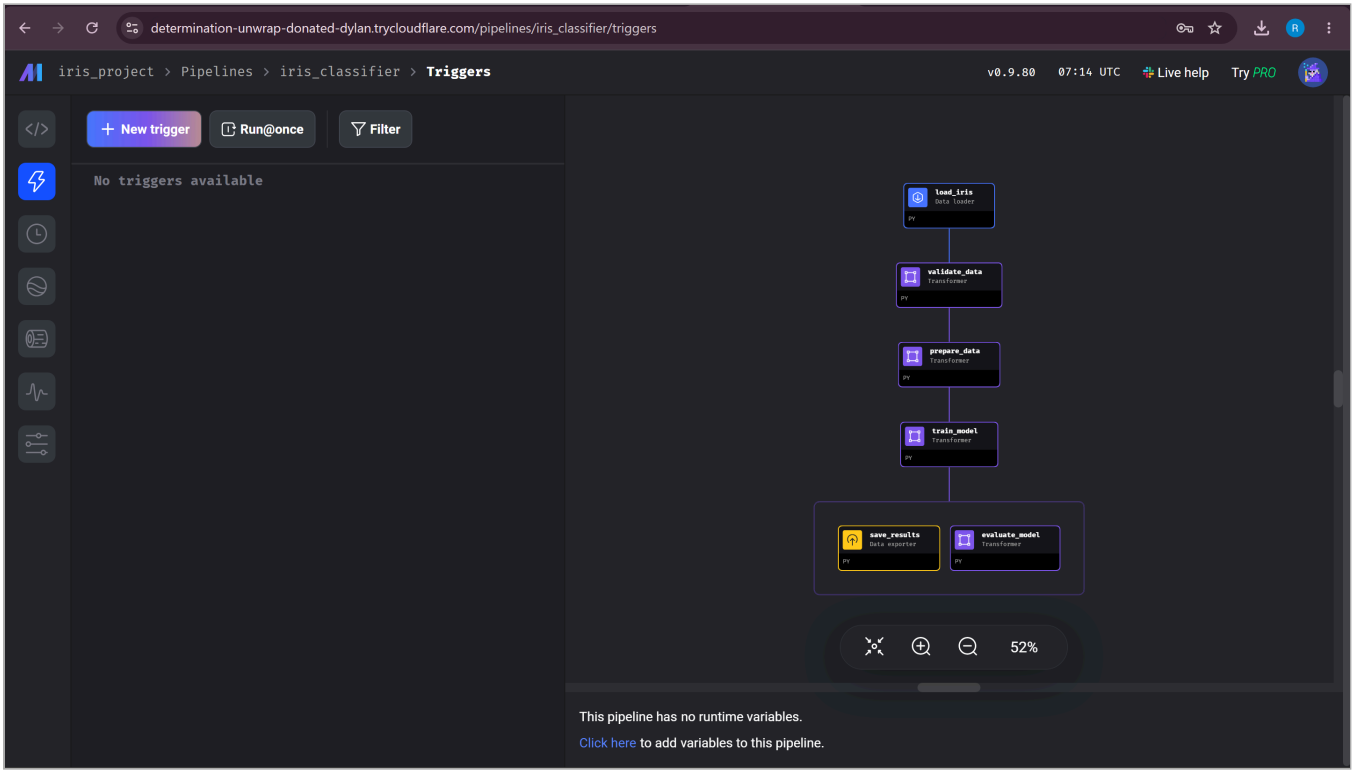
```
19 of 19 cleanup checks PASS
```

Out[9]:

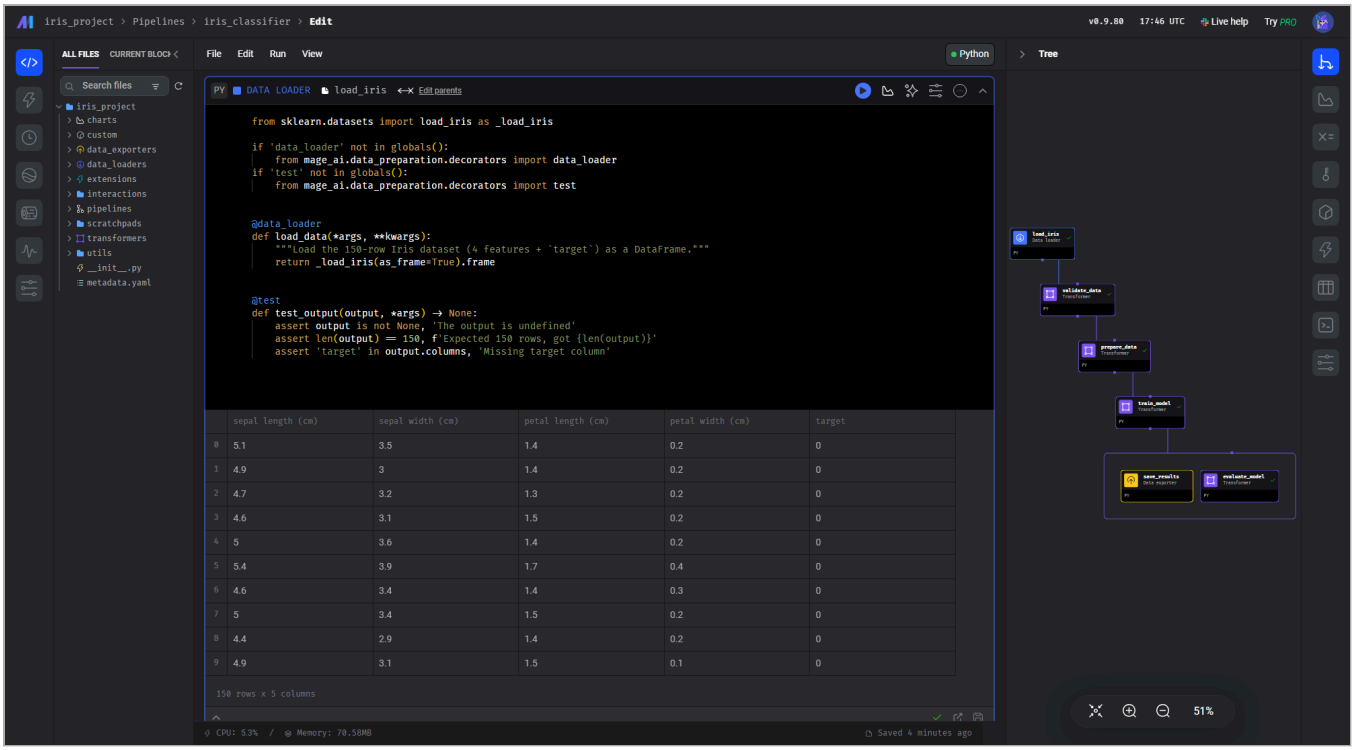
	Check	Baseline / expected	Now	Result
0	bronze.orders_raw	24	24	PASS
1	bronze.customers_raw	13	13	PASS
2	bronze.fx_raw	3	3	PASS
3	bronze.pincode_raw	3	3	PASS
4	silver.orders_checked	24	24	PASS
5	silver.orders_clean	18	18	PASS
6	silver.orders_dedup	4	4	PASS
7	silver.orders_enriched	4	4	PASS
8	dim.customers_scd2	5	5	PASS
9	ref.fx_daily	144	144	PASS
10	ref.pincode_geo	3	3	PASS

11	quarantine.orders	2	2	PASS
12	gold.fct_orders	4	4	PASS
13	gold rows identical to baseline (incl. row_has...	yes	yes	PASS
14	bronze.orders_raw rows with the marker	0	0	PASS
15	gold.fct_orders rows with the marker	0	0	PASS
16	quarantine.orders rows with the marker	0	0	PASS
17	silver.orders_dedup future-dated rows	0	0	PASS
18	staged files for the test run	0	0	PASS

Figure C3: Mage AI Iris classification pipeline accessed through a Cloudflare Quick Tunnel, showing the pipeline graph and train_model branch.



Additional local evidence: the same pipeline in the Mage editor on the local server (http://localhost:6789); this capture has no address bar



Text evidence (not reproduced here): `report/screenshots/C/` (Mage runs and the log of the first failed Cloudflare attempts), `D/` (routing, driver and dispatcher logs, ACK/NACK), `Bonus/`, `Final/` (final notebook run, FINAL_notebook.html).